While I can't directly output a literal `.pdf` file download, I have structured this entire breakdown exactly like a high-quality, professional technical document. You can easily print this page to PDF or save it to your notes.

Let's cut through the theory and look at this from a purely practical, engineering perspective. When you are building LLM agents, getting memory right is the difference between a toy project and a production-ready system.

Here is your comprehensive breakdown of the 3-Part Agent Memory Architecture.

AI Agent Memory Architecture: The 3-Part Framework

1. The Core Problem: Why "Obvious" Fixes Fail

When an AI agent "forgets" things, engineers typically try two quick fixes. Both fail in production:

1. **Context Window Stuffing (The "Bigger Desk" Problem):** Feeding the entire conversation history into the LLM prompt.

- *The Result:* Latency spikes, costs skyrocket, and the model suffers from "attention drift" (focusing on irrelevant old data instead of the current task).

2. **The Single Vector Store (The "Junk Drawer" Problem):** Dumping chat history, facts, and documents into one vector database.

- *The Result:* The database returns data based on *meaning* (semantic similarity), ignoring *time* (recency) or *authority* (hard facts). It might return an old, canceled account plan just because the wording closely matched the prompt.

The Diagnosis: Memory is not a *storage* problem; it is a *retrieval* problem. Different types of data require entirely different retrieval strategies.

---

2. The Jargon Dictionary: Core Definitions

To understand the architecture, let's break down the technical terminology used in this space into simple terms.

- **Context Window:** The maximum amount of text (measured in tokens) an LLM can process in a single request. Think of it as the model's short-term RAM.
- **Vector Database (Vector Store):** A database that stores information as arrays of numbers (embeddings) and retrieves them based on conceptual meaning rather than exact keyword matches.
- **Embeddings:** The conversion of text into numerical vectors so a machine can measure how closely related two concepts are.
- **Cosine Similarity / Semantic Similarity:** The mathematical formula used by vector databases to measure how closely related two pieces of text are. It measures "closeness in meaning," but it completely ignores *when* something happened or if it's factually correct.
- **Entity / Entity Extraction:** An "Entity" is a specific noun—like a user ID, a company name, or a product tier. "Extraction" is the process of pulling that exact noun out of a user's prompt (often using simple Regex or basic NLP) so you can look up hard facts about it.
- **Behavioral Scaffold / Exemplar:** A verified, successful example of how a task was completed in the past. It acts as a template (scaffold) for the agent to follow so it doesn't hallucinate a new, incorrect workflow.

---

3. The 3-Memory Framework Breakdown

Instead of one giant database, a production agent splits data into three distinct stores, exactly like the human brain.

I. Episodic Memory (The "Event Log")

Definition: The record of *what happened and when*. It tracks the history of interactions, context, and user preferences over time.

- **When to use it:** When the agent needs to know about past conversations, recent decisions, or changes in user preference.
- **Where to use it:** In chat interfaces, customer support bots, or any session-based interaction where the user might change their mind.
- **For which task:** "I changed my mind, actually go with the plan I mentioned last Tuesday."

- **How it works (The Python Logic):** You cannot rely on similarity alone. You must write a Python function that blends **Semantic Relevance** (is this related to the query?) with **Temporal Recency** (did this happen recently?). You apply an *exponential decay factor* so older memories are scored lower, unless they are flagged as highly "important" (like a final decision).

II. Semantic Memory (The "Fact Engine")

Definition: The authoritative storage of structural facts about the world, the user, or the business.

- **When to use it:** When you need a hard, undeniable truth that should not be guessed or fuzzily matched.
- **Where to use it:** Looking up API endpoints, checking a user's current subscription tier in a CRM, or retrieving system constraints.
- **For which task:** "What is the data limit on my current account?"
- **How it works (The Python Logic):** Do **not** use standard vector search here. Use **Entity-First Retrieval**. Extract the entity (e.g., `user_id: 12345`) from the prompt, and do a direct database lookup in a graph or relational database. If an old episodic memory conflicts with a semantic fact (e.g., the user *said* they are on Pro, but the database *says* they are on Basic), you use priority rules to decide which wins. (Usually, the system CRM wins for facts, but episodic wins for user preferences).

III. Procedural Memory (The "Skill Library")

Definition: The knowledge of *how* to execute a specific process or use tools correctly.

- **When to use it:** When the agent needs to perform an action, trigger an API, or complete a multi-step workflow.
- **Where to use it:** Agentic systems that have access to tools (like sending emails, executing code, or querying external databases).
- **For which task:** "Generate a weekly report for Acme Corp and email it to the admin."
- **How it works (The Python Logic):** You store **Exemplars**—which are verified, 100% successful logs of this task being done correctly in the past. When the user asks to do a task, the agent retrieves the exemplar and uses it as a template. *Critical Rule:* Never record failed task attempts into this memory, or the agent will learn how to fail.

At a Glance: The 3 Memories

Feature	Episodic (Blue)	Semantic (Green)	Procedural (Amber)
Core Question	What happened?	What is true?	How do I do it?
Data Stored	Event logs, conversations, timestamps.	Facts, entities, account data, rules.	Skill templates, successful tool sequences.
Retrieval Strategy	Semantic Relevance + Recency Decay	Direct Entity Lookup (Exact Match)	Exemplar retrieval for task scaffolding

---

4. The Memory Router (The Dispatcher)

You don't query all three databases blindly every time. You build a lightweight **Memory Router** (a classification layer) that reads the user's prompt and decides which store to hit.

- **Temporal keywords** (*"last time", "before", "previously"*): Routes to **Episodic**.
- **Factual keywords** (*"what is", "who is", "which tier"*): Routes to **Semantic**.
- **Action keywords** (*"create", "run", "build", "fix"*): Routes to **Procedural**.

If the prompt is ambiguous and has no clear keywords, the router triggers a "Parallel Fusion Fallback"—it queries all three stores simultaneously and lets the LLM figure it out.

5. Production Metrics: How to know it's working

When deploying this architecture, you track four main metrics to ensure your agent is actually job-ready:

1. **Memory Hit Rate (>60%):** Are your queries actually finding relevant data? If this is low, your data isn't being stored correctly in the first place.
2. **Ambiguous Routing (<20%):** If more than 20% of user prompts are bypassing your specific routers and hitting the "fallback" query, your Python classification logic needs tighter keyword rules.
3. **Recency (< 30 Days):** If episodic memory is pulling up chat logs from 3 months ago for a current problem, your math for the temporal decay factor is too weak.

4. **Procedural Success Rate:** Are tasks succeeding more often when using your retrieved exemplars compared to when the agent just wings it? If not, your exemplars are contaminated with bad data.